TABLE OF CONTENTS

Chapter	Title	Page No.
Chapter 1	**INTRODUCTION**	**1**
1.1	Characteristics and Properties	1
1.2	Types of Algorithms Advantages	2
1.3	and Disadvantages Designing an	3
1.4	Algorithm Description of the	4
1.5	Project	4
Chapter 2	**REQUIREMENT SPECIFICATIONS**	5
2.1	Hardware Requirements	5
2.2	Software Requirements	5
2.3	Functional Requirements	5
Chapter 3	**SYSTEM DESIGN**	6
3.1	Algorithm	6
3.2	Basic Working	7-8
Chapter 4	**IMPLEMENTATION**	9-10
Chapter 5	**RESULTS**	11
Chapter 6	**CONCLUSION AND FUTURE ENHANCEMENTS**	12
Chapter 7	**REFERENCES**	13

Chapter 1

INTRODUCTION

A **Minimum Spanning Tree (MST)** is an important concept in graph theory used to connect all vertices in a weighted graph with the **minimum possible total edge cost** and without forming any cycles. It is widely used in network design problems such as building computer networks, road systems, and communication networks.

Kruskal's Algorithm is a greedy algorithm used to find the Minimum Spanning Tree of a connected, weighted graph. The algorithm works by selecting the **smallest weight edges first** and adding them to the spanning tree, while ensuring that no cycles are formed.

In this project, Kruskal's Algorithm is implemented to optimize network connections by selecting edges with the lowest cost. By sorting the edges based on their weights and adding them step by step while avoiding cycles, the algorithm efficiently constructs the **minimum cost spanning tree** for the given graph.

Thus, Kruskal's Algorithm helps in achieving **network optimization** by reducing the overall connection cost while maintaining connectivity between all nodes.

1.1 Characteristics and Properties

- It is a **greedy algorithm** used to find the Minimum Spanning Tree.
- It **selects edges with the smallest weight first**.
- All edges are **sorted in increasing order of weight** before processing.
- It **avoids cycles** while adding edges to the spanning tree.
- It works on **weighted and undirected graphs**.
- It uses the **Union-Find (Disjoint Set)** method to detect cycles.
- The final Minimum Spanning Tree contains **V − 1 edges** for **V vertices**.
- It **connects all vertices with minimum total cost**.
- It is **efficient for sparse graphs** (graphs with fewer edges).
- The **time complexity is O(E log E)**.

Properties :

• Greedy algorithm that selects the smallest weight edge first.

• Used to find the **Minimum Spanning Tree (MST)** of a graph.

• Works on **undirected weighted graphs**.

• Edges are **sorted in increasing order of weight**.

• Prevents **cycles** while adding edges.

• Uses **Union-Find (Disjoint Set)** to check cycles.

• MST contains **V − 1 edges** for **V vertices**.

• Produces **minimum total edge weight**.

• For disconnected graphs, it forms a **Minimum Spanning Forest**.

• Time complexity is **O(E log E)**.

1.2 Types

• **Kruskal's Algorithm** – Builds the MST by selecting the smallest weight edges and avoiding cycles.

• **Prim's Algorithm** – Starts from a vertex and expands the tree by adding the minimum weight edge.

• **Borůvka's Algorithm** – Repeatedly adds the shortest edge from each component to build the MST.

1.3 Advantages and Disadvantages

Advantages :

- Simple and easy to understand.
- Efficient for **sparse graphs** (graphs with fewer edges).
- Guarantees **minimum total cost** for connecting all vertices.
- Uses **greedy approach**, so it selects optimal edges step-by-step.
- Can handle **disconnected graphs** by forming a minimum spanning forest.
- Works well with **Union–Find data structure** for cycle detection.
- Time complexity is relatively efficient **O(E log E)**.

Disadvantages :

- Edge sorting increases the **time complexity**.
- Not very efficient for **dense graphs** (graphs with many edges).
- Requires additional **memory for Union–Find data structure**.
- Cycle detection adds **extra processing steps**.
- Not suitable for graphs where edges are **frequently changing .**

Kruskal's Algorithm is simple and easy to understand because it follows a greedy approach that always selects the edge with the smallest weight. It is efficient for sparse graphs and guarantees a minimum total cost for connecting all vertices in a network. The algorithm also works well with the Union–Find data structure to detect cycles and can handle disconnected graphs by forming a minimum spanning forest. However, Kruskal's Algorithm has some disadvantages. It requires sorting all edges before processing, which increases the time complexity. It is also less efficient for dense graphs with many edges, and cycle detection using additional data structures can require extra memory and processing time.

1.4 Designing an Kruskal's Algorithm

Kruskal's Algorithm is designed using a greedy approach to find the Minimum Spanning Tree (MST) of a weighted graph. First, all the edges in the graph are arranged in increasing order of their weights. Then the algorithm selects the smallest edge and adds it to the spanning tree if it does not create a cycle. To check whether a cycle is formed, a Union–Find (Disjoint Set) data structure is used. This process continues by repeatedly selecting the next smallest edge and adding it to the tree. The algorithm stops when the spanning tree contains V − 1 edges, where V is the number of vertices in the graph. The final result is a tree that connects all vertices with the minimum possible total weight.

1.5 Description of the Project

Kruskal's Algorithm is used to find the Minimum Spanning Tree (MST) of a connected, weighted, and undirected graph. The algorithm works by selecting the edge with the smallest weight and adding it to the spanning tree. Before selecting edges, all edges of the graph are sorted in increasing order of their weights. Each edge is then checked to ensure that adding it does not form a cycle in the graph. If it does not create a cycle, the edge is included in the MST; otherwise, it is discarded. This process continues until the spanning tree contains V − 1 edges, where V is the number of vertices. The final result is a tree that connects all the vertices with the minimum total edge weight.

- Kruskal's Algorithm is used to find the Minimum Spanning Tree (MST) of a weighted graph.
- It follows a greedy approach by selecting the smallest edge first.
- The graph must be undirected and weighted.
- All edges are sorted in increasing order of their weights.
- The smallest edge is selected and added to the MST.
- Before adding an edge, the algorithm checks whether it forms a cycle.
- Union–Find (Disjoint Set) is used to detect cycles.
- If adding the edge creates a cycle, the edge is discarded.
- The process continues until V − 1 edges are included in the tree.
- The final result connects all vertices with the minimum total weight.

Chapter 2

REQUIREMENT SPECIFICATIONS

A requirement specification is a document that describes the requirements ofthe software product, program or set of programs.

2.1 Hardware Requirements

- Computer or Laptop
- Minimum 4 GB RAM
- Minimum 20 GB free disk space
- Processor: Intel i3 or higher

2.2 Software Requirements

- Operating System: Windows / Linux / macOS
- Programming Language: Python / C / C++ / Java
- Code Editor: VS Code / PyCharm / Turbo C
- Compiler or Interpreter for running the program

2.3 Functional Requirements

- Accept input for vertices and edges of the graph
- Allow user to enter weights for each edge
- Sort edges based on weight
- Apply Kruskal's Algorithm to find MST
- Display the edges of the Minimum Spanning Tree
- Show the total minimum cost of the MST

Chapter 3

SYSTEM DESIGN

• Input Design

- The system accepts input of vertices and edges of the graph.
- The user enters source vertex, destination vertex, and edge weight.
- Input should be clear and validated to avoid errors.

• Process Design

- The system sorts all edges in increasing order of weight.
- It applies Kruskal's Algorithm to select edges with minimum weight.
- Cycle detection is performed using the Union–Find (Disjoint Set) method.
- Edges are added to the Minimum Spanning Tree (MST) only if they do not create a cycle.

• Output Design

- The system displays the edges included in the MST.
- It also shows the total minimum cost of the spanning tree.
- Output should be clear and easy to understand.

3.1 Algorithm

Step 1: Start the program.

Step 2: Input the number of vertices (V) and edges (E) in the graph.

Step 3: Enter all edges with their source vertex, destination vertex, and weight. Step 4:

Sort all edges of the graph in ascending order of their weights.

Step 5: Create a disjoint set (Union–Find) for all vertices to track connected components. Step 6:

Initialize an empty set to store the Minimum Spanning Tree (MST) edges.

Step 7: Select the edge with the smallest weight from the sorted list.

Step 8: Check whether adding this edge forms a cycle using the Union–Find method. Step 9:

- If no cycle is formed, add the edge to the MST.
- If a cycle is formed, discard the edge.

Step 10: Repeat Steps 7–9 until the MST contains V − 1 edges.

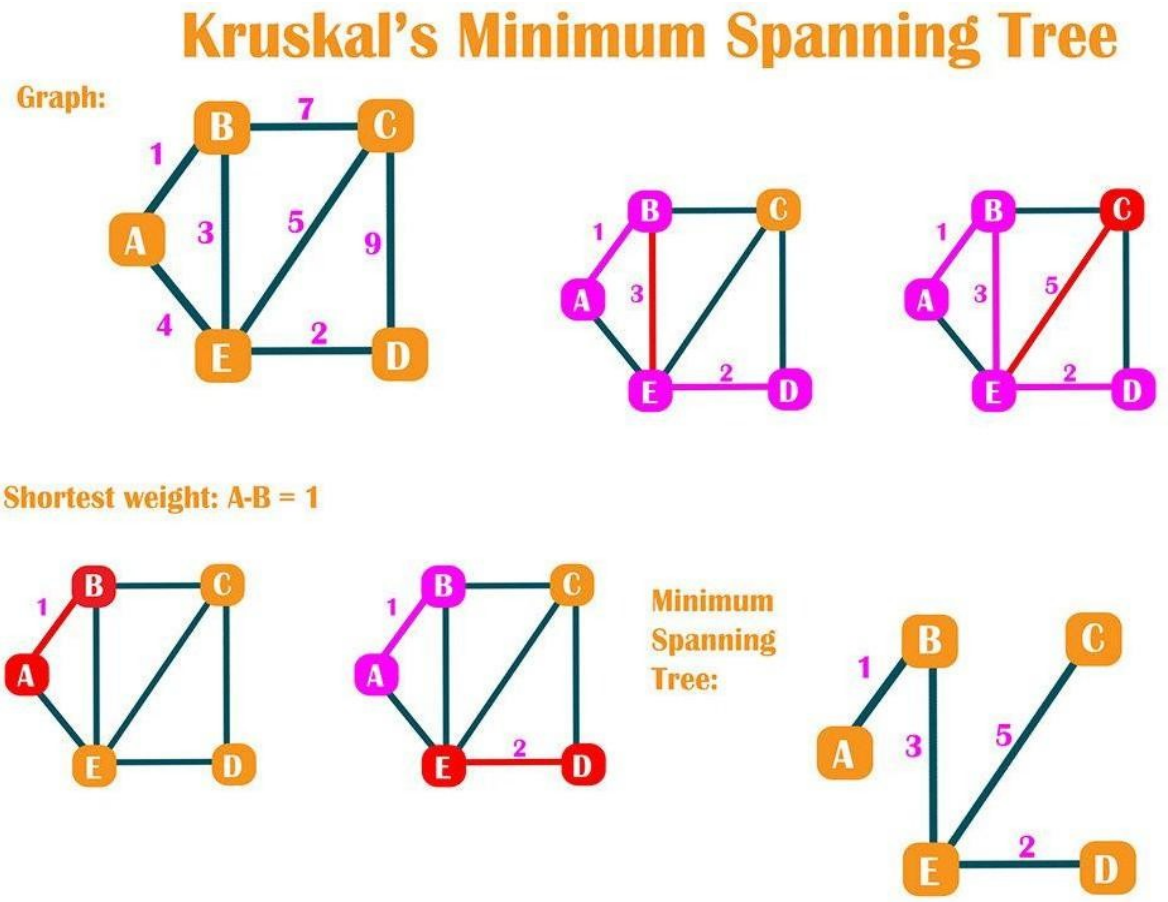
Step 11: Display the edges of the Minimum Spanning Tree and the total minimum cost. Step 12:

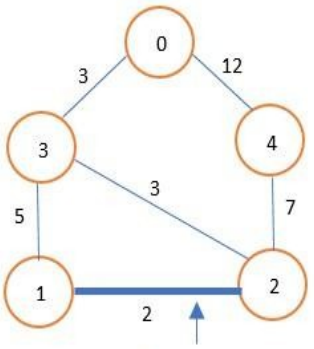
Stop the program.

3.2 Basic Working

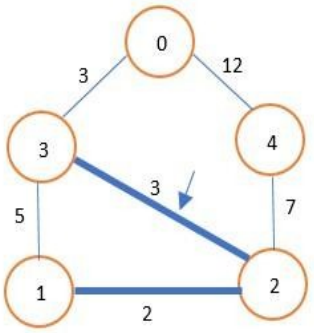
- The algorithm starts by taking a weighted, undirected graph as input.
- All the edges of the graph are sorted in increasing order of their weights.
- The algorithm then selects the edge with the smallest weight.
- Before adding the edge, it checks whether it forms a cycle with the already selected edges.
- If the edge does not create a cycle, it is added to the Minimum Spanning Tree (MST).
- If it creates a cycle, the edge is ignored.
- This process continues by selecting the next smallest edge.
- The algorithm stops when V − 1 edges are added, where V is the number of vertices.
- Finally, the result is a Minimum Spanning Tree with the smallest possible total weight.

For example:

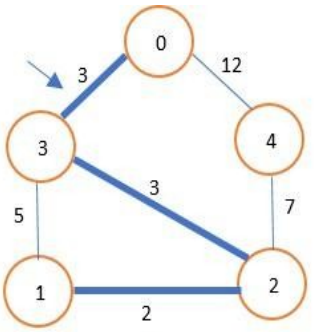



0
3
12
3
4
3
5
7
1
2
2

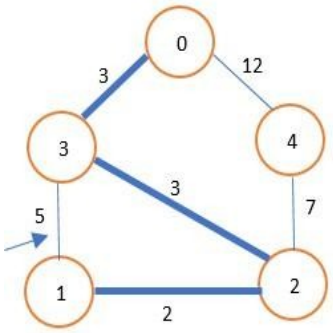
1


0
3
12
3
4
3
5
7
1
2
2

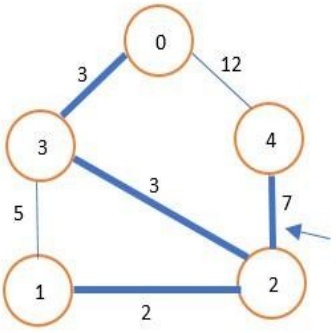
2


0
3
12
3
4
3
5
7
1
2
2

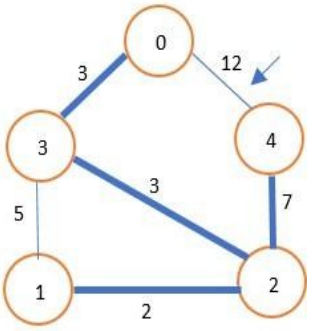
3


0
3
12
3
4
3
5
7
1
2
2

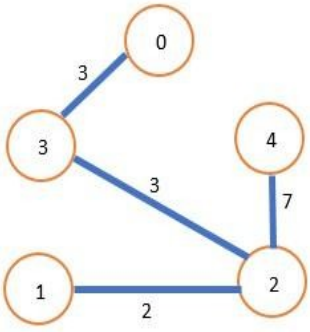
4


0
3
12
3
4
3
5
7
1
2
2

5


0
3
12
3
4
3
5
7
1
2
2

6


0
3
3
4
7
1
2
2

7

Chapter 4

IMPLEMENTATION

```
#include <stdio.h>
#include <stdlib.h>

#define MAX 20

// Structure to store an edge struct
Edge {
   int u, v, w;
};

// Parent array for Union-Find int
parent[MAX];

// Function to find parent (for cycle detection) int
find(int i) {
   while (parent[i] != i) i
      = parent[i];
   return i;
}

// Function to perform union of two sets void
unionSet(int a, int b) {
   parent[a] = b;
}

// Function to sort edges based on weight void
sortEdges(struct Edge edges[], int e) {
   int i, j;
   struct Edge temp;

   for (i = 0; i < e - 1; i++) {
      for (j = i + 1; j < e; j++) {
         if (edges[i].w > edges[j].w) {
            temp = edges[i];
            edges[i] = edges[j];
            edges[j] = temp;
         }
      }
   }
}

int main() {
   int v, e, i;
   struct Edge edges[MAX];

   printf("Enter number of vertices: "); scanf("%d",
   &v);
```

```
printf("Enter number of edges: "); scanf("%d",
&e);

// Input edges
for (i = 0; i < e; i++) {
    printf("Enter source, destination and weight for edge %d: ", i + 1); scanf("%d %d
    %d", &edges[i].u, &edges[i].v, &edges[i].w);
}

// Initialize parent array for
(i = 0; i < v; i++)
    parent[i] = i;

// Sort edges by weight
sortEdges(edges, e);

printf("\nEdges in the Minimum Spanning Tree:\n"); int

count = 0, cost = 0;

// Process edges
for (i = 0; i < e && count < v - 1; i++) { int
    u = find(edges[i].u);
    int v2 = find(edges[i].v);

    // If adding edge does not form a cycle if
    (u != v2) {
        printf("%d -- %d == %d\n", edges[i].u, edges[i].v, edges[i].w); cost
        += edges[i].w;
        unionSet(u, v2); count++;
    }
}

printf("Total cost of MST = %d\n", cost); return

0;
```

Chapter 5

RESULTS

Output

```
Enter number of vertices: 4
Enter number of edges: 5
Enter source, destination and weight for edge 1: 0 1 10
Enter source, destination and weight for edge 2: 0 2 6
Enter source, destination and weight for edge 3: 0 3 5
Enter source, destination and weight for edge 4: 1 3 15
Enter source, destination and weight for edge 5: 2 3 4

Edges in the Minimum Spanning Tree:
2 -- 3 == 4
0 -- 3 == 5
0 -- 1 == 10
Total cost of MST = 19


=== Code Execution Successful ===
```

Chapter 6

CONCLUSION AND FUTURE ENHANCEMENTS

Kruskal's Algorithm is an effective technique used to determine the Minimum Spanning Tree (MST) of a weighted and undirected graph. It follows a greedy strategy by always choosing the edge with the smallest weight while ensuring that no cycles are formed. This helps in connecting all vertices of the graph with the minimum possible total cost. The algorithm is simple to understand and implement, and it works efficiently for sparse graphs. Because of these advantages, Kruskal's Algorithm is widely used in applications such as network design, road and railway planning, electrical circuit design, and communication systems. Overall, the algorithm provides an optimal solution for minimizing the cost of connecting multiple nodes in a network.

Future Enhancements

• Add a Graphical User Interface (GUI) to make the program more interactive and user-friendly.

• Include graph visualization to show how the Minimum Spanning Tree is built step by step.

• Extend the system to support larger and more complex graphs.

• Implement additional algorithms such as Prim's Algorithm and Borůvka's Algorithm for comparison.

• Improve performance using more efficient data structures and optimized sorting methods.

• Integrate the program with real-world datasets for practical network analysis.

• Allow users to import graph data from files such as CSV or text files.

• Add error handling and validation for better reliability.

• Develop a web-based version so that users can run the algorithm online.

• Provide performance analysis and comparison charts between different algorithms.

Chapter 7

REFERENCES

• Introduction to Algorithms – Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein.

• Data Structures and Algorithm Analysis in C – Mark Allen Weiss.

• GeeksforGeeks – Articles on Kruskal's Algorithm and Minimum Spanning Tree.

• TutorialsPoint – Explanation and examples of Kruskal's Algorithm.

• Wikipedia – Information on Minimum Spanning Tree and Kruskal's Algorithm.

• IEEE publications on graph algorithms and network optimization.

Data Structures and Algorithms Made Easy – Narasimha Karumanchi.

• Algorithms – Robert Sedgewick and Kevin Wayne.

• The Design and Analysis of Computer Algorithms – Alfred V. Aho, John E. Hopcroft and Jeffrey D. Ullman.

• Stack Overflow – Discussions and solutions related to algorithm implementation.

• W3Schools – Basic concepts of algorithms and programming.

• Khan Academy – Educational resources for algorithms and graph theory.